RAG no Projeto Nina

Como o chatbot consulta documentos para responder perguntas

Este guia explica o funcionamento do RAG e sua aplicação no protótipo do Projeto Nina, voltado ao apoio informativo a mulheres em situação de vulnerabilidade. O objetivo é apresentar o caminho entre os PDFs, a busca de informações e a resposta exibida no site.

O que significa RAG

RAG é a sigla de Retrieval Augmented Generation, ou geração aumentada por recuperação. A técnica combina a busca de informações em uma base externa com a geração de texto por um modelo de linguagem. Neste projeto, o sistema procura trechos de PDFs e os fornece ao Qwen como contexto para responder [1].

Uma analogia útil é uma prova com consulta: a pergunta chega, o sistema procura páginas relacionadas e o modelo redige uma resposta com o material encontrado. A consulta ajuda a fundamentar a resposta, mas não garante que ela esteja correta.

Os três elementos principais

Elemento	Papel no projeto
Base de conhecimento	Textos extraídos dos PDFs escolhidos para consulta.
Recuperação	Modelo de embeddings e FAISS localizam trechos próximos do significado da pergunta.
Geração	Qwen recebe a pergunta, os trechos e as instruções para escrever a resposta.

RAG não é treinamento do modelo

Adicionar PDFs à base não altera os pesos do Qwen. Os documentos são processados e indexados para serem consultados durante a conversa. O modelo de linguagem já foi treinado anteriormente; aqui ele recebe informações adicionais no momento da resposta.

Por que usar no Projeto Nina

A proposta é orientar as respostas por documentos selecionados pelo projeto e permitir identificar arquivo e página de origem. Isso facilita a revisão do conteúdo e a atualização da base. O protótipo deve ser apresentado como apoio informativo, sem prometer diagnóstico, proteção ou substituição de atendimento profissional.

Como a informação percorre o sistema

Primeira etapa preparar a base

Essa etapa foi realizada no Colab. Ela precisa ser refeita quando os documentos ou o modelo de embeddings mudarem, e não a cada pergunta.

1. Selecionar os PDFs e verificar a qualidade, a procedência e a atualização do conteúdo.
2. Extrair o texto das páginas. PDFs que são apenas imagens podem exigir reconhecimento de texto por OCR.
3. Dividir o texto em trechos menores, chamados chunks, mantendo a referência ao arquivo e à página.
4. Transformar cada trecho em um embedding, uma lista de números que representa características semânticas do texto.
5. Armazenar os vetores em um índice FAISS e salvar os textos, os metadados e a configuração.

Segunda etapa responder no site

6. A pessoa digita uma pergunta na página HTML. O JavaScript envia a mensagem ao servidor Flask.
7. O Flask chama a função de resposta em rag_backend.py.
8. O mesmo modelo de embeddings usado na indexação transforma a pergunta em um vetor.
9. O FAISS compara esse vetor com o índice e recupera os trechos mais semelhantes. Na versão fornecida, são até cinco trechos.
10. O código monta um prompt com instruções, trechos recuperados e pergunta. O Qwen gera a resposta.
11. O Flask devolve o resultado ao JavaScript, que o apresenta no chat.

Exemplo para explicar em sala

Pergunta ilustrativa: “O que significa ciclo da violência?” O sistema busca trechos relacionados a esse assunto, inclui esses trechos no contexto e solicita ao Qwen uma explicação. Se o material recuperado sustentar a resposta, o modelo deve citar o documento e a página. Este é um exemplo do processo, não um resultado de teste desta pergunta.

Busca por significado

A busca vetorial pode relacionar expressões diferentes sobre um mesmo assunto. Por exemplo, “controlar minhas amizades” pode se aproximar de um trecho sobre isolamento. A similaridade é uma medida de proximidade entre vetores; ela não representa a probabilidade de a resposta estar correta.

Componentes do protótipo

A descrição considera o código fornecido nesta conversa e o ZIP da base verificado. Alterações feitas posteriormente no computador podem mudar o comportamento.

Arquivo ou componente	Responsabilidade
app.py e Flask	Receber as requisições do site, chamar o RAG e devolver respostas ou erros.
rag_backend.py	Carregar modelos e base, buscar trechos e gerar a resposta.
HTML CSS e JavaScript	Organizar a página, definir sua aparência e enviar e exibir mensagens.
chunks.pkl	Guardar os trechos de texto extraídos dos documentos.
indice.faiss	Guardar o índice de vetores usado na busca.
metadados.json	Relacionar os trechos aos nomes de arquivos e às páginas.
config.json	Registrar modelo de embeddings, dimensão e quantidades da base.
embeddings.npy	Guardar os vetores gerados. O backend fornecido consulta o índice FAISS e não precisa ler esse arquivo diretamente.
requirements.txt	Listar as dependências Python necessárias para executar o projeto.

Modelos e base utilizada

Embeddings: sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2. No ZIP enviado, cada vetor possui 384 dimensões. A base contém 53 trechos, provenientes de ManualVersãoNova.pdf e ManualRevisado.pdf. Esses números descrevem a base inspecionada; não medem sua qualidade ou cobertura.

Geração: Qwen/Qwen2.5-1.5B-Instruct, modelo de linguagem ajustado para seguir instruções [2]. O modelo de embeddings faz a representação para busca; o Qwen escreve a resposta. São funções diferentes.

Por que a primeira pergunta demora

O backend carrega a base e os modelos na primeira pergunta. Caso os modelos ainda não estejam disponíveis no computador, também pode ocorrer download. Depois do carregamento, o código reutiliza os objetos na memória enquanto o processo continua ativo.

O tempo de geração depende do hardware e do tamanho do contexto e da resposta. A configuração automática de dispositivo não comprova uso da GPU. A versão fornecida devolve o texto completo ao final da geração, sem transmissão gradual dos tokens.

Limites e roteiro de apresentação

O que precisa ser avaliado

RAG pode recuperar trechos inadequados e o modelo pode interpretar mal o contexto ou inventar informações. A versão fornecida busca os cinco trechos mais próximos sem um limiar mínimo de similaridade. Portanto, mesmo uma pergunta fora do assunto pode receber contexto pouco relevante. A instrução para admitir falta de informação ajuda, mas não garante a recusa.

Uma avaliação útil reúne perguntas com respostas esperadas e páginas de referência. É preciso verificar se a busca encontra os trechos certos, se cada afirmação é sustentada pelas fontes, se o sistema admite informação insuficiente e quanto tempo leva para responder. Não há uma taxa de acerto medida neste material.

Para esse tema, a seleção e a atualização dos documentos são essenciais. Citações devem ser conferidas. Relatos pessoais não devem ser publicados no repositório nem incorporados automaticamente à base. O arquivo chunks.pkl contém texto dos documentos e deve vir de uma origem confiável.

Roteiro de fala de aproximadamente três minutos

“No Projeto Nina, utilizamos RAG para conectar o chatbot a uma base de documentos sobre o tema do projeto. RAG significa geração aumentada por recuperação. A ideia é buscar informações relevantes antes de produzir uma resposta.

Primeiro, preparamos a base no Colab. Extraímos o texto dos PDFs, dividimos esse texto em pequenos trechos e transformamos cada trecho em uma representação numérica, chamada embedding. Esses vetores são organizados em um índice FAISS, junto de referências ao arquivo e à página.

Quando uma pessoa envia uma pergunta pelo site, o JavaScript encaminha a mensagem para o Flask. O servidor chama o código do RAG, que transforma a pergunta em um vetor e busca os trechos mais semelhantes. Em seguida, esses trechos são enviados ao Qwen junto com a pergunta e as instruções de resposta.

É importante diferenciar as funções: o modelo de embeddings ajuda a buscar informações; o Qwen gera o texto. Os PDFs não treinam novamente o Qwen. Eles servem como material de consulta no momento em que a resposta é produzida.

A principal vantagem é aproximar a resposta de documentos selecionados e permitir conferir a origem das informações. Porém, isso não elimina erros. Precisamos testar a recuperação, conferir as fontes e avaliar perguntas que a base não consegue responder. O Projeto Nina é um protótipo de apoio informativo que exige validação antes de uso real.”

Referências e origem das informações

[1] LEWIS, Patrick et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. 2020. https://arxiv.org/abs/2005.11401

[2] QWEN. Qwen2.5-1.5B-Instruct — documentação do modelo. https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

Implementação: rag_backend.py fornecido na conversa e inspeção do arquivo base_rag.zip enviado pelo autor. Consulta em 19 de outubro de 2026.